

Internship Training Program – Week 3

Prepared By

Adnan Mushtaq (Web Lead)

Duration

1 Week

Tech Stack

Node.js, Express.js, SQLite, Prisma ORM, JWT, bcrypt, Multer, Streams, Crypto, MVC Architecture

Repository

Public GitHub Repository (Continue using feature branches)

Program Overview – Week 3

This week focuses on **Backend Engineering Fundamentals** using modern Node.js architecture and production-ready patterns.

The focus is on:

- Node.js core concepts
- Express.js architecture
- Database integration with Prisma
- Authentication & Authorization
- File handling & Streams
- Clean MVC structure
- Secure backend development
- Scalable API design

Each day includes:

- In-depth theory
- Practical implementation
- Daily GitHub feature branch push

Git Rules (Same as Week 1)

Main branch: `main`

No direct commits to main

Branch Naming Convention:

```
feat/week2-day1-node-core  
feat/week2-day2-prisma-db  
feat/week2-day3-auth-rbac  
feat/week2-day4-file-upload  
feat/week2-day5-final-project
```

Recommended Backend Folder Structure

```
src/  
├─ controllers/  
├─ routes/  
├─ services/  
├─ middlewares/  
├─ models/  
├─ prisma/  
├─ utils/  
├─ config/  
├─ uploads/  
└─ app.js
```

Day 1 – Node.js Core + Express + MVC

1. Node.js Core Concepts

Topics Covered

Node.js Architecture

- Event loop
- Non-blocking I/O
- Single-threaded model

Core Modules

FS Module

- readFile
- writeFile
- readFileSync
- createReadStream
- createWriteStream

Stream Module

- Readable
- Writable
- Duplex
- Transform
- Piping streams

Crypto Module

- Hashing (SHA256)
- Password hashing basics
- Random token generation

Util Module

- promisify()
- Converting callbacks → promises

2. Modules System

CommonJS

```
const fs = require("fs")
module.exports = {}
```

ES Modules

```
import fs from "fs"
export default {}
```

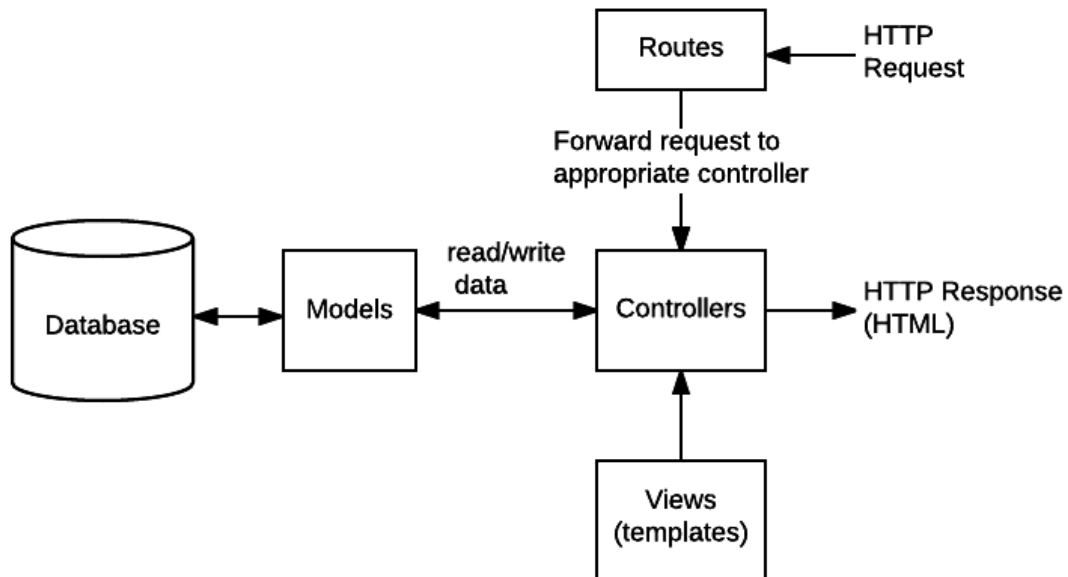
Comparison:

- require vs import
- module.exports vs export
- Default vs named exports
- Node config: `"type": "module"`

Internal vs External Modules:

- Internal → your own files
- External → npm packages

3. Express.js Introduction



Express Basics

- App initialization
- Routing
- Middleware concept
- Request & Response lifecycle

Express Middlewares

- Built-in (express.json)
- Third-party
- Custom middleware
- Error handling middleware

MVC Architecture

- Model
- Controller
- Route
- Separation of concerns

Day 1 – Hands-On Implementation

Build a basic Express API with MVC:

Requirements

- Setup Express server
- Create `/api/files/read`
- Read a large file using `createReadStream`
- Return file as stream to client
- Add logging middleware
- Use crypto to hash a sample string

Day 1 Task

Task:

Build a File Streaming API

Requirements:

- Create a sample text file (large)
- Read file using Node.js stream
- Return response as stream
- Do NOT use `readFileSync`
- Use MVC structure
- Push to `feat/week2-day1-node-core`

Day 2 – Database Integration (SQLite + Prisma ORM)

Topics Covered

1 Database Basics

- Relational database
- Tables
- Primary key
- Foreign key
- Relationships

2 SQLite

- Lightweight database
- File-based storage

3 Prisma ORM

- Schema definition
- Models
- Migrations
- Prisma Client

Example Schema:

```
model User {  
  id    Int    @id @default(autoincrement())  
  name  String  
  email String  @unique  
  role  String  @default("USER")  
}
```

4 CRUD Operations

- Create
- Find many
- Find unique
- Update
- Delete

Day 2 – Hands-On

Integrate Prisma into existing Express app.

Add:

- User model
- Product model
- Connect SQLite
- Create REST endpoints

Day 2 Task

Task:

Extend Day 1 App with Database

Requirements:

- Setup Prisma
- Create User model
- Implement:
 - POST /users
 - GET /users
 - GET /users/:id
 - DELETE /users/:id
- Proper error handling
- Push to [feat/week2-day2-prisma-db](#)

Day 3 – Authentication + Authorization (RBAC)

Topics Covered

1 Authentication

- What is authentication?
- Stateless auth
- JWT structure

2 Password Security

- Hashing using bcrypt
- Never store plain passwords

3 JWT

- Sign token
- Verify token
- Token expiration

4 Authorization

- Role-based access control (RBAC)
- Middleware-based role checking

Example Roles:

- ADMIN
- USER

Day 3 – Hands-On

Add authentication to existing app.

Features:

- Register
- Login
- Password hashing
- JWT token generation
- Protect routes
- Admin-only route

Day 3 Task

Task:

Implement Authentication System

Requirements:

- POST /auth/register
- POST /auth/login
- Protect /users route

- Add role-based middleware
 - Only ADMIN can delete users
 - Push to [feat/week2-day3-auth-rbac](#)
-

Day 4 – File Upload + Streams + Buffers

Topics Covered

1 Multer

- Disk storage
- Memory storage
- File filtering
- Limits

Memory vs Disk:

- Memory → stored in buffer
- Disk → saved in uploads folder

2 Static Files

```
app.use("/uploads", express.static("uploads"))
```

3 Buffers

- What is a buffer?
- Binary data handling

4 Stream Handling

- Large file upload
- Pipe stream to storage

Day 4 – Hands-On

Add file upload to existing authenticated app.

Day 4 Task

Task:

Add Profile Image Upload

Requirements:

- Authenticated upload
- Save file to /uploads
- Store file path in database
- Return image via static route
- Add file size validation
- Push to `feat/week2-day4-file-upload`

Day 5 – Final Project (Backend API System)

Final Project: Intern Management Backend API

Build a complete backend system combining everything learned.

Required Features

Authentication:

- Register
- Login
- JWT
- Role-based access

Database:

- Users
- Interns
- Tasks

CRUD:

- Create Intern
- Assign Task
- Update Status
- Delete (Admin only)

File Upload:

- Intern profile image
- Task attachment

Streaming:

- Download task attachment via stream

Security:

- Password hashing
- Role-based middleware
- Proper error handling

Day 5 Task

Task:

Build Full Intern Management API

Requirements:

- Complete MVC structure
- Prisma + SQLite
- JWT Auth
- Role-based control
- File upload
- Stream file download
- Clean code structure
- Proper commits
- Push to `feat/week2-day5-final-project`

Evaluation Criteria – Week 2

- Understanding of Node.js core modules
- Proper use of streams
- Clean Express MVC architecture
- Correct Prisma usage
- Secure authentication implementation
- File handling efficiency
- Role-based authorization
- Git discipline
- Code structure & scalability